



Chapter 11 Object-Oriented Programming: Inheritance



©2025, College of Software Engineering, Southeast University

OBJECTIVE

- To create classes by **inheriting** from existing classes(通过继承现有类来创建新类)
- How inheritance promotes software reuse(继承如何提高软件重用)
- The concepts of **base classes** and **derived classes** and relationships between them(基类和派生类的概念及二者之间关系)
- The **protected** member access specifier(protected成员访问说明符)
- The use of constructors and destructors in **inheritance hierarchies**(继承层次中构造函数和析构函数的用法)
- The difference between **public**, **protected** and **private** inheritance (public, protected和private继承的差异)
- The use of inheritance to customize existing software(使用继承定制软件)

©2025, College of Software Engineering, Southeast University

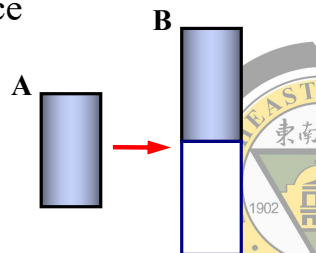
11.1 Introduction(cont.)

• Definition of inheritance

- Create new class from existing class

- Absorb the data and behaviors of existing class(es)
- Enhance with new capabilities

- *Derived(son) class* inherits from *base(parent) class*



Class A derives Class B
 • Class A is *base class* (father class)
 • Class B is *derived class* (son class)

©2025, College of Software Engineering, Southeast University

3

11.2 Base Classes and Derived Classes

• Inheritance Relationship

※ **is-a** relation (Inheritance)

Square is a rectangle;
Teacher is a person

※ **has-a** relation (Composition)

※ Base class / Derived class

※ Direct/Indirect base class

※ Single/Multiple inheritance

※ 3 kinds of inheritance

- public
- protected
- private

©2025, College of Software Engineering, Southeast University

4

11.2 Base Classes and Derived Classes

- Base class (基类): 被继承的类
- Derived class (派生类): 继承后得到的类

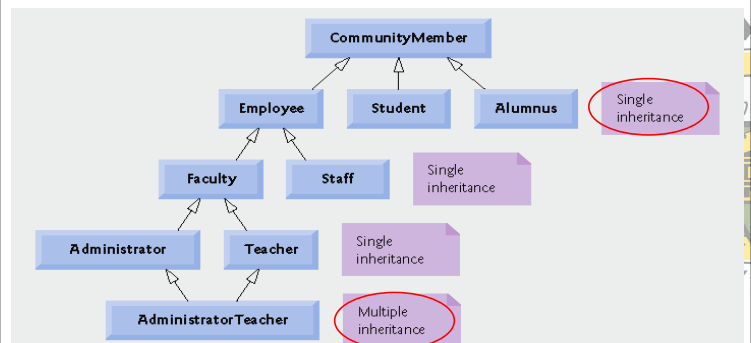
Base class	Derived classes
Student	GraduateStudent, UndergraduateStudent
Shape	Circle, Triangle, Rectangle, Sphere, Cube
Loan	CarLoan, HomeImprovementLoan, MortgageLoan
Employee	Faculty, Staff
Account	CheckingAccount, SavingsAccount

©2025, College of Software Engineering, Southeast University

5

11.2 Base Classes and Derived Classes (cont.)

- Inheritance hierarchy: **Tree-like** hierarchy structure



继承机制的主要作用: 软件复用, 支持系统的增量开发。

©2025, College of Software Engineering, Southeast University

6

11.2 Base Classes and Derived Classes (cont.)

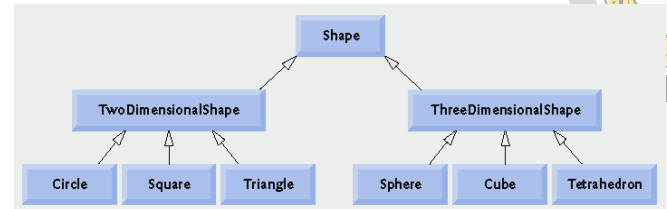
• Member composition of derived class

- Inherited members
 - All members of base class except **constructors**, **destructors**, and **friend functions**
- New additional members
 - Including constructor and destructor of derived class

11.2 Base Classes and Derived Classes (cont.)

• Inheritance Syntax

```
class TwoDimensionalShape : public Shape
{
    // New additional members
}
```



11.2 Base Classes and Derived Classes (cont.)

• Base class members can be redefined in derived class with the same name

- Base class members are **hidden** in derived class
- Base class members can be accessed in derived class by

```
BaseClassName :: MemberName
BaseClassName :: MemberName(<...>)
```

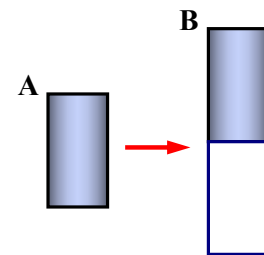
• Redefined members include

- Data member: same name
- Member function: same name (returnType, params number/type/order may be same or not)

11.2 Base Classes and Derived Classes (cont.)

• 3 kinds of inheritance:

public protected private(default)



```
class B : public A{...};
class B : protected A{...};
class B : private A{...};
class B : A{...};
```

A is private derived

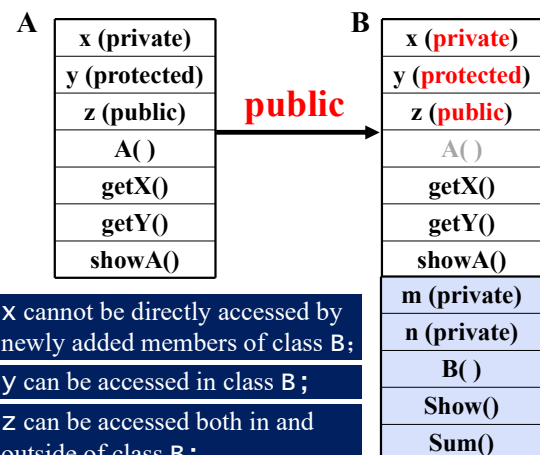
11.2 Base Classes and Derived Classes (cont.)

(1) Public Inheritance

```
class ClassName: public BaseClassName
```

All the inherited members **maintain** their access authority in base class to derived class.

Base class access authority	Derived class access authority	In derived class	Outside of derived class
public	public	accessible	accessible
protected	protected	accessible	inaccessible
private	private	inaccessible	inaccessible



- x cannot be directly accessed by newly added members of class B;

- y can be accessed in class B;

- z can be accessed both in and outside of class B;

- Initialize an object of class B → Initialize x, y, z, m, n

11.2 Base Classes and Derived Classes (cont.)

(2) private Inheritance

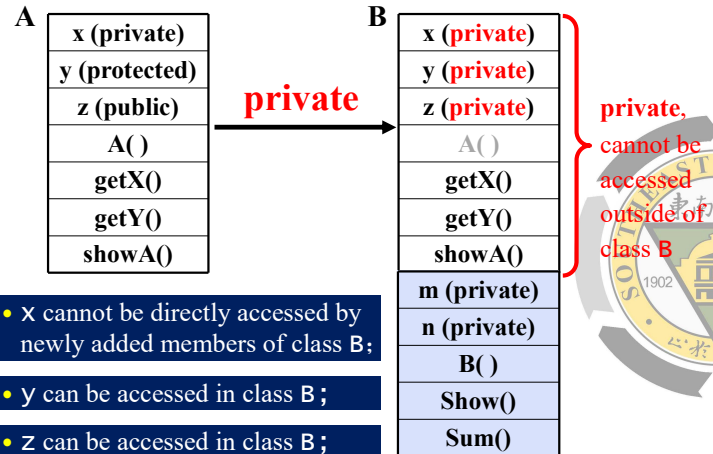
class ClassName: **private** BaseClassName

Inherited member with **Public** or **protected** authority in base class changes into **private** in derived class.

Base class access authority	Derived class access authority	In derived class	Outside of derived class
public	private	accessible	inaccessible
protected	private	accessible	inaccessible
private	private	inaccessible	inaccessible

©2025, College of Software Engineering, Southeast University

13



©2025, College of Software Engineering, Southeast University

14

11.2 Base Classes and Derived Classes (cont.)

(3) protected Inheritance

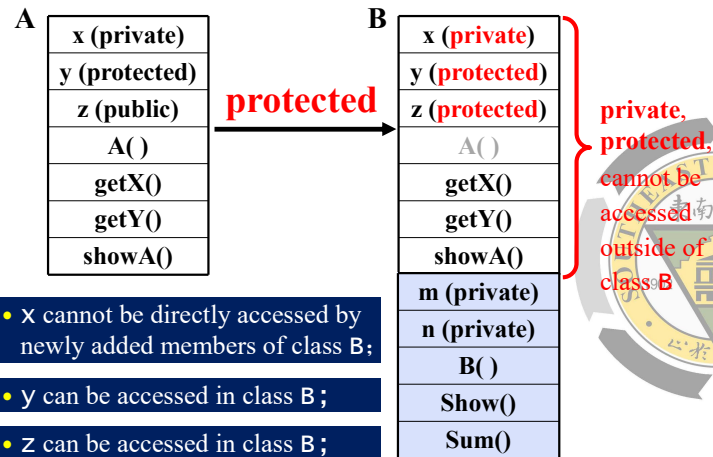
class ClassName: **protected** BaseClassName

Inherited member with **public** or **protected** authority in base class changes into **protected** in derived class.

Base class access authority	Derived class access authority	In derived class	Outside of derived class
public	protected	accessible	inaccessible
protected	protected	accessible	inaccessible
private	private	inaccessible	inaccessible

©2025, College of Software Engineering, Southeast University

15



©2025, College of Software Engineering, Southeast University

16

Topics

- 11.1 Introduction
- 11.2 Base Classes(基类) and Derived Classes (派生类)
- 11.3 protected Members
- **11.4 Relationship between Base Classes and Derived Classes**
- 11.5 Constructors and Destructors in Derived Classes
- 11.6 public, protected and private Inheritance
- 11.7 Software Engineering with Inheritance

©2025, College of Software Engineering, Southeast University

11.4 Relationship between Base Classes and Derived Classes

- ※ **Commission Employee** 佣金雇员
- ※ **Base-salaried commission employees** 带底薪的佣金雇员

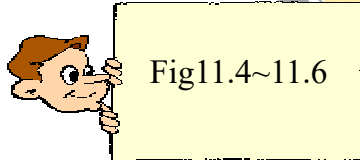
- 11.4.1 Creating and Using a **CommissionEmployee** Class
- 11.4.2 Creating a **BasePlusCommissionEmployee** Class Without Using Inheritance
- 11.4.3 Creating a CommissionEmployee-BasePlusCommissionEmployee **Inheritance** Hierarchy
- 11.4.4 CommissionEmployee-BasePlusCommissionEmployee Inheritance Hierarchy Using **protected Data**
- 11.4.5 CommissionEmployee-BasePlusCommissionEmployee Inheritance Hierarchy Using **private Data**

©2025, College of Software Engineering, Southeast University

18

11.4.1 Creating and Using a CommissionEmployee Class

- class `CommissionEmployee`
(first name, last name, social security number, commission rate, gross sales amount)

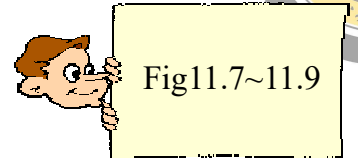


©2025, College of Software Engineering, Southeast University

19

11.4.2 Creating a BasePlusCommissionEmployee Class Without Using Inheritance

- class `BasePlusCommissionEmployee`
(first name, last name, social security number, commission rate, gross sales amount, **base salary**)
- class `CommissionEmployee`
(first name, last name, social security number, commission rate, gross sales amount)

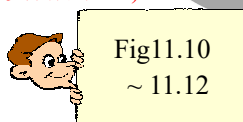


©2025, College of Software Engineering, Southeast University

20

11.4.3 Creating a CommissionEmployee-BasePlusCommissionEmployee Inheritance Hierarchy

- Class `BasePlusCommissionEmployee`
 - Derived from class `CommissionEmployee`
 - Is a `CommissionEmployee`
 - Inherits all members (except constructor and destructor)
 - Constructor
 - Use **base class initializer syntax**(基类初始化器) to initialize base class data member
 - New data member `baseSalary`

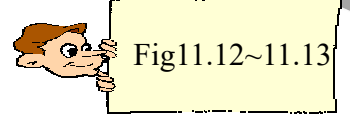


©2025, College of Software Engineering, Southeast University

21

11.4.4 CommissionEmployee-BasePlusCommissionEmployee Inheritance Hierarchy Using protected Data

- Use protected data
 - Enable class `BasePlusCommissionEmployee` to directly access base class data members
- A base class's **protected** members can be accessed by members and **friends** of the base class itself and those of any classes derived from it.



©2025, College of Software Engineering, Southeast University

22

Notices on protected data

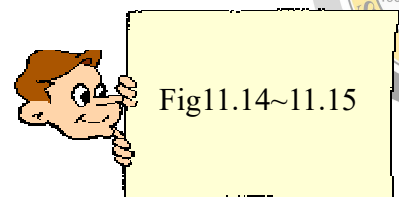
- Using protected data members
 - Advantages
 - Derived class can modify values directly
 - Slight increase in performance
 - Disadvantages
 - No validity checking
 - Derived class can assign illegal value
 - Implementation dependent
 - Base class implementation changes may result in derived class modifications
 - » Fragile (brittle) software

©2025, College of Software Engineering, Southeast University

23

11.4.5 CommissionEmployee-BasePlusCommissionEmployee Inheritance Hierarchy Using private Data

- Reexamine hierarchy
`private` data members + `public` function interfaces



©2025, College of Software Engineering, Southeast University

24

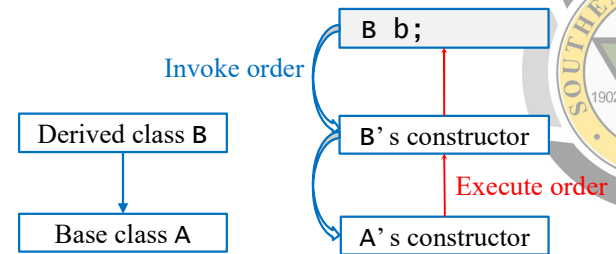
11.5 Constructors and Destructors in Derived Classes

- Derived class does not inherit its base class's constructors and destructors, **but can invoke them**
- When an derived class's object is constructed or destroyed
 - Constructors or destructors of both **derived class** and **base class** will be invoked

©2025, College of Software Engineering, Southeast University 25

11.5 Constructors and Destructors in Derived Classes(cont.)

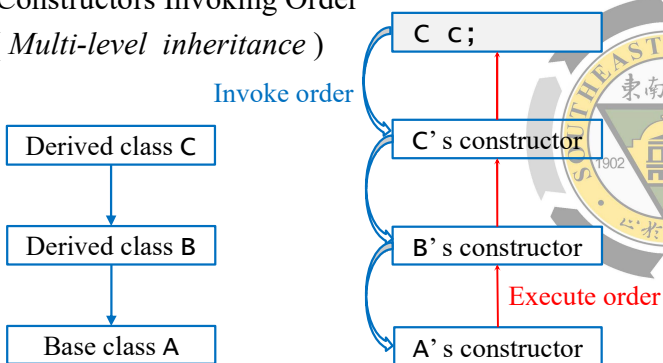
- Constructors Invoking Order



©2025, College of Software Engineering, Southeast University 26

11.5 Constructors and Destructors in Derived Classes(cont.)

- Constructors Invoking Order
(Multi-level inheritance)



©2025, College of Software Engineering, Southeast University 27

11.5 Constructors and Destructors in Derived Classes(cont.)

- Invoke base class's constructors via
 - Default constructor (*Implicitly*)
 - Base class member initializer (*Explicitly*)

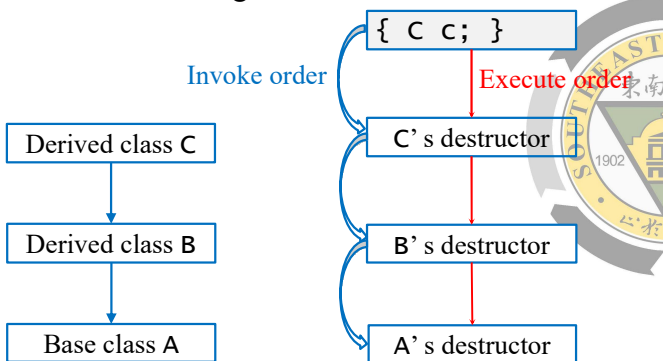
```

10 // constructor
11 BasePlusCommissionEmployee::BasePlusCommissionEmployee (
12     const string &first, const string &last, const
13     string &ssn,
14     double sales, double rate, double salary )
15     // explicitly call base class constructor
16     : CommissionEmployee ( first, last, ssn, sales,
17                             rate)
18 {
19     setBaseSalary ( salary ); // validate and store
20 }

```

11.5 Constructors and Destructors in Derived Classes(cont.)

- Destructor Invoking Order



©2025, College of Software Engineering, Southeast University 29

```

int main()
{
    cout << fixed << setprecision( 2 )
    { // begin new scope
        CommissionEmployee employee1(
            "Bob", "Lewis", "333-33-3333"
        ) // end scope

        cout << endl;
        BasePlusCommissionEmployee
            employee2("Lisa", "Jones", "555-55-5555"

        cout << endl;
        BasePlusCommissionEmployee
            employee3("Mark", "Sands", "888-88-8888"
        cout << endl;
        return 0;
    }
}

```

CommissionEmployee constructor:
 CommissionEmployee destructor:
 CommissionEmployee constructor:
 BasePlusCommissionEmployee constructor:
 CommissionEmployee constructor:
 BasePlusCommissionEmployee constructor:
 BasePlusCommissionEmployee destructor:
 CommissionEmployee destructor:
 BasePlusCommissionEmployee destructor:
 CommissionEmployee destructor:

Block {

employee3
BasePlusCommissionEm
p
employee
CommissionEmployee

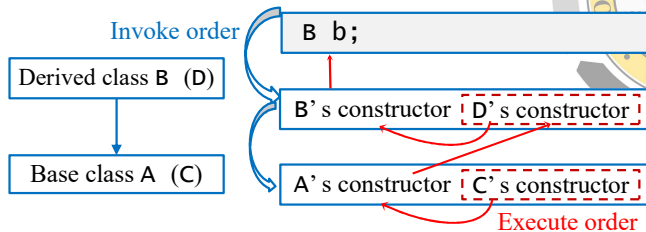
employee2
BasePlusCommissionEm
p
employee
CommissionEmployee

employee1

©2025, College of Software Engineering, Southeast University 30

11.5 Constructors and Destructors in Derived Classes(cont.)

- When classes contain (via *composition*) objects of other classes
 - Construct: 先基类后派生类, 先被包含类后宿主类
 - Destruct: 相反



©2025, College of Software Engineering, Southeast University

31

11.7 Software Engineering with Inheritance

- 派生类的程序员不需了解基类的源代码, 只需与目标代码连接即可
 - ISV (独立软件供应商) 为目标代码提供头文件, 发放许可。
 - 继承实用的类库, 提高软件复用。
- 软件工程提示:
 - 提取出共同的属性和行为并把它们放在一个基类中, 然后通过继承生成派生类。
 - 尽量建立较少的类, 避免不必要的类。

©2025, College of Software Engineering, Southeast University

32

Summary

- Definitions of Base class and Derived class
- protected member
- Relations between Base class and Derived class: public, protected, private
- Constructors and Destructors in Derived Classes

©2025, College of Software Engineering, Southeast University

33